SIMATS Engineering

SIMATS Online Programming Lab — Python Programming

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO2-AT2-SCENARIO BASED ASSESSMENT
Question	Q3 — Scenario 3: Structured Server Log Error Analyser (20 Marks)

Scenario 3: Structured Server Log Error Analyser [20 Marks]

A server continuously generates log files containing various system messages. These logs include informational, warning, and error messages in text format. The system must extract only error-related entries for analysis. String searching techniques should be used to identify relevant lines. The program should count occurrences of different types of errors. It must also remove duplicate error messages for concise reporting. Control flow should manage filtering and classification of log entries. Functions must be used to modularize extraction and summarization tasks.

Task

Develop a Python program to perform structured log error analysis and reporting, demonstrating the following key skills:

- String searching: in, find(), startswith()
- Filtering using for loop and conditionals
- Duplicate removal logic
- Error counting and categorization
- Functions for extraction, filtering, and summarization

Approach / Design

• **Extraction (extract_error_lines):** loops through every log line and uses the 'in' operator together with str.find() to identify lines containing the ERROR keyword.

• **Message extraction (get_error_message):** uses find() to locate the ERROR marker and slices out the descriptive message that follows it.

• **Duplicate removal (remove_duplicate_errors):** loops over the extracted error lines and keeps only the first occurrence of each distinct error message using a seen-set, producing a concise report.

• **Classification (classify_error):** uses startswith() and the 'in' operator inside if-elif-else logic to categorize each error message (Database, NullPointerException, Timeout, Disk write failure, Other).

• **Counting (count_error_types):** loops over all extracted error lines and tallies how many fall into each category.

• **Summarization (summarize_logs):** orchestrates extraction, counting, and duplicate removal into a single summary dictionary.

• **Reporting (print_report):** prints total logs scanned, total errors found, per-category counts, and the deduplicated error list.

Python Program

```
"""
CO2-AT2 - Scenario Based Assessment
Scenario 3: Structured Server Log Error Analyser
------------------------------------------------------------
A server continuously generates log files containing INFO,
WARNING, and ERROR messages. This program:
  1. Extracts only error-related entries using string searching
     (in, find(), startswith())
  2. Filters lines using a for loop and conditionals
  3. Removes duplicate error messages for concise reporting
  4. Counts occurrences of different error types
  5. Is fully modularized using functions for extraction,
     filtering, and summarization
"""

# ------------------------------------------------------------
# 1. Sample raw log data (list of log lines)
# ------------------------------------------------------------
RAW_LOGS = [
    "2026-08-01 09:00:12 INFO Server started successfully",
    "2026-08-01 09:02:45 WARNING Disk usage at 80%",
    "2026-08-01 09:05:10 ERROR Database connection failed",
    "2026-08-01 09:05:12 ERROR Database connection failed",
    "2026-08-01 09:07:33 INFO User admin logged in",
    "2026-08-01 09:10:01 ERROR NullPointerException in module Auth",
    "2026-08-01 09:12:20 WARNING High memory usage detected",
    "2026-08-01 09:15:44 ERROR Database connection failed",
    "2026-08-01 09:18:02 ERROR Timeout while connecting to API",
    "2026-08-01 09:20:15 ERROR NullPointerException in module Auth",
    "2026-08-01 09:22:30 INFO Backup completed",
    "2026-08-01 09:25:47 ERROR Disk write failure",
]

# Recognized error categories to classify against
ERROR_CATEGORIES = [
    "Database connection failed",
    "NullPointerException",
```

```
    "Timeout",
    "Disk write failure",
]


# ---------------------------------------------------------------
# 2. Extraction function — finds error-related lines
# ---------------------------------------------------------------
def extract_error_lines(logs):
    """
    Uses string searching (the 'in' operator and str.find()) to
    identify and extract only log lines that are error entries.
    """
    error_lines = []
    for line in logs:
        # 'in' operator for keyword search
        if "ERROR" in line:
            # find() confirms position & demonstrates the method explicitly
            if line.find("ERROR") != -1:
                error_lines.append(line)
    return error_lines


# ---------------------------------------------------------------
# 3. Message extraction helper
# ---------------------------------------------------------------
def get_error_message(log_line):
    """
    Extracts the descriptive message portion of an error log line
    (the text after the 'ERROR' keyword).
    """
    marker = "ERROR"
    idx = log_line.find(marker)
    return log_line[idx + len(marker):].strip()


# ---------------------------------------------------------------
# 4. Duplicate removal function
# ---------------------------------------------------------------
def remove_duplicate_errors(error_lines):
    """
    Filters out duplicate error messages (based on the message
    text, ignoring timestamps) using a loop and a seen-set check,
    preserving the first occurrence order for concise reporting.
    """
    seen_messages = set()
    unique_lines = []

    for line in error_lines:
        message = get_error_message(line)
        if message not in seen_messages:
            seen_messages.add(message)
            unique_lines.append(line)

    return unique_lines


# ---------------------------------------------------------------
# 5. Classification / categorization function
# ---------------------------------------------------------------
def classify_error(message):
    """
    Classifies an error message into a known category using
    string searching and if-elif-else control flow. Falls back
    to 'Other' for unrecognized errors.
    """
    if message.startswith("Timeout"):
        return "Timeout"
    elif "Database connection failed" in message:
```

```
        return "Database connection failed"
    elif "NullPointerException" in message:
        return "NullPointerException"
    elif "Disk write failure" in message:
        return "Disk write failure"
    else:
        return "Other"


# ---------------------------------------------------------------
# 6. Error counting function (based on ALL errors, incl. duplicates)
# ---------------------------------------------------------------
def count_error_types(error_lines):
    """
    Counts occurrences of each error category across all extracted
    error lines (including duplicates), using a loop and
    conditional classification.
    """
    counts = {}
    for line in error_lines:
        message = get_error_message(line)
        category = classify_error(message)
        counts[category] = counts.get(category, 0) + 1
    return counts


# ---------------------------------------------------------------
# 7. Summarization function
# ---------------------------------------------------------------
def summarize_logs(logs):
    """
    Orchestrates the full pipeline: extraction, counting, and
    duplicate removal, returning a summary dictionary.
    """
    error_lines = extract_error_lines(logs)
    error_counts = count_error_types(error_lines)
    unique_errors = remove_duplicate_errors(error_lines)

    return {
        "total_logs": len(logs),
        "total_errors": len(error_lines),
        "unique_errors": unique_errors,
        "error_counts": error_counts,
    }


# ---------------------------------------------------------------
# 8. Report/display function
# ---------------------------------------------------------------
def print_report(summary):
    print("=" * 70)
    print("SERVER LOG ERROR ANALYSIS REPORT")
    print("=" * 70)

    print(f"\nTotal log entries scanned : {summary['total_logs']}")
    print(f"Total error entries found : {summary['total_errors']}")

    print("\nError counts by category:")
    for category, count in summary["error_counts"].items():
        print(f"  {category:<30}: {count}")

    print(f"\nUnique error entries ({len(summary['unique_errors'])}) for concise reporting:")
    for idx, line in enumerate(summary["unique_errors"], start=1):
        print(f"  {idx}. {line}")


# ---------------------------------------------------------------
# 9. Main driver
# ---------------------------------------------------------------
```

```
def main():
    summary = summarize_logs(RAW_LOGS)
    print_report(summary)


if __name__ == "__main__":
    main()
```

Sample Output

```
======================================================================
SERVER LOG ERROR ANALYSIS REPORT
======================================================================

Total log entries scanned : 12
Total error entries found : 7

Error counts by category:
  Database connection failed    : 3
  NullPointerException          : 2
  Timeout                       : 1
  Disk write failure            : 1

Unique error entries (4) for concise reporting:
  1. 2026-08-01 09:05:10 ERROR Database connection failed
  2. 2026-08-01 09:10:01 ERROR NullPointerException in module Auth
  3. 2026-08-01 09:18:02 ERROR Timeout while connecting to API
  4. 2026-08-01 09:25:47 ERROR Disk write failure
```

Conclusion

The program isolates error entries from mixed server logs using string searching (in, find(), startswith()), removes duplicate messages so reports stay concise, and classifies each error into a category using if-elif-else control flow. Because extraction, deduplication, classification, and counting are each wrapped in their own function and driven by loops over the log list, the design cleanly separates concerns and scales to any volume of log data.